

RCS-VRF Leg 1 — Seed Generation

A primer on what we are adopting, what we are rejecting, and why

Design note · Leg 1 (seed) · for internal alignment

The decision in one line

RCS-VRF Leg 1 generates the round seed from a **hash-based, multi-source canonical transcript**—not from any verifiable random function (VRF). The seed’s unpredictability rests on a hash function and the assumption that *at least one* contributing source is honest. There is no secret key whose secrecy the seed depends on.

What we are going with

The seed for each round is the hash of a canonical transcript combining four independent components. Compromising the seed’s unpredictability requires defeating *all* of them; a single honest contributor suffices.

- **Commit–reveal** — operator and public-pool participants commit to values before the round and reveal afterward, so no party can choose its contribution after seeing others’.
- **External beacons** — **drand** and **NIST**, two independently operated public randomness beacons, each contributing a pulse.
- **Hash-based public puzzle** — a proof-of-work-style puzzle anyone can solve, providing permissionless liveness so the round can advance without privileged participants.
- **Chained transcript** — a `prev_finalized_round_hash` field linking each round to the previous finalized one, making the history tamper-evident.

Why this shape. The construction needs only *public inputs and a hash*. There is no key to distribute, no threshold ceremony, and no standing network to operate. Unpredictability is an *any-one-honest* property: an adversary controlling three of four sources still cannot predict or steer the output.

Scope note. Signing keys still exist in the design—but only for *attribution* (non-repudiation of commits; verifying beacon-pulse authenticity). They are *not* load-bearing for unpredictability. “No VRF” does not mean “no elliptic-curve crypto anywhere”; it means no secret key sits on the unpredictability path.

What we are not going with — and why

EC-VRF (the primitive)

A VRF output is a deterministic function of a public input and a **secret key**. The construction is publicly *verifiable*, but its *unpredictability* depends entirely on the key staying secret. This is the root defect, and it cascades:

- **The keyholder controls every output.** Given the key, all outputs are computable in advance; the holder can predict them and—depending on workflow—selectively publish or withhold. Security collapses to *trusting that party*, the precise assumption RCS-VRF exists to remove.
- **Classically secure only.** EC-VRF rests on elliptic-curve hardness (discrete log), which a quantum adversary breaks. A hash-based seed carries no such exposure.
- **It is only a seed.** At best a VRF substitutes for Leg 1 alone. It says nothing about Legs 4–6, where the actual certified *quantum* entropy is injected and certified. Treating a classical VRF as a substitute for the whole protocol is a category error: a VRF yields publicly-verifiable *pseudo*-randomness; RCS-VRF yields *certified quantum* entropy.

Chainlink VRF (the productized service)

Chainlink VRF is built on EC-VRF, so it inherits every problem above and adds deployment-level ones:

- **Single keyholder** — a single point of trust, versus our any-one-of-four-honest model.
- **Request-then-fulfill workflow** — a window opens between requesting randomness and the response landing, giving the fulfiller an MEV / withholding opportunity. Our schedule-driven design has no such window.

The multi-participant VRF workaround — and its cost

Distributing the key across many holders (threshold / aggregated VRF, as in Polkadot BABE or drand) does fix the *cryptographic* defect: no single party can predict or control outputs. But it cannot do so for free—it requires standing up and operating a validator / federation network (key distribution, threshold signing, liveness, incentives). That overhead is only justifiable when the network *already exists for another reason* (Polkadot runs validators for consensus; drand’s federation is a public good), in which case you *consume* the byproduct rather than build it. Building such a network for randomness alone is not justifiable.

Our design sidesteps the dilemma entirely: **no key means no single point of trust *and* no network to operate**. We still lean on drand / NIST as *sources*, but we do not *operate* any validator network, and we do not depend on any single one of them being honest.

Side-by-side

	Single EC-VRF (Chainlink)	Threshold VRF (BABE / drand)	Hash multi-source (RCS-VRF Leg 1)
Fixes single-keyholder trust	No	Yes	Yes
Secret key on unpredictability path	Yes	Yes (distributed)	No
Validator network to operate	—	Yes (free only if pre-existing)	No
Manipulation window	Yes (request/fulfill)	No (scheduled)	No (scheduled)
Quantum-secure seed	No	No (VRF / BLS)	Yes (hash-based)

The framing to carry into any conversation

“We’re not building a VRF. We’re solving the same problem a VRF solves, with a primitive that has no secret whose secrecy the seed’s unpredictability depends on—and which is quantum-secure. A VRF could at most feed Leg 1; the certified entropy comes from the quantum chain downstream.”

Open items to confirm with Mehdi: (1) the canonical source set on the seed transcript (slides 18–19) as the externally-quoted version; (2) umbrella naming—“QVRF” is now misleading, working label “QSeed” is narrower than the deck’s true scope; tying the name to the *output* (Certified Randomness) may be cleanest.